

Scalehouse

An API-first AI phone receptionist for service businesses: the product, how it works, the stack it runs on, and what production needs.



Repository

ScaleHouse (main @ 57f8af5)

Runtime

Node.js 20+ · TypeScript

Prepared

September 27, 2026

Contents

1 Executive summary

What Scalehouse is, who uses it, and where it stands

2 Product overview

The problem, the actors, and the value chain from phone call to CRM

3 Technology stack

Languages, frameworks, and managed services, and the reasons for each

4 System architecture

Components, boundaries, and how the code is organized

5 Inbound call flow

The latency-critical assistant-request path and the generated receptionist

6 Completed-call pipeline

Persistence, AI analysis, owner SMS, and the idempotent state machine

7 Customer integration API

Authentication, scopes, endpoints, errors, and rate limiting

8 Outbound webhooks

Events, signing, verification, and retry behaviour

9 Data model

Tables, constraints, and database functions, by migration

10 Security model

Tenant isolation, credential handling, and logging hygiene

11 Infrastructure & deployment

Topology, hosting requirements, schedulers, and configuration

12 Operations

Operator CLI, go-live checklist, and troubleshooting

13 Testing & quality

How the code is verified today

14 Current state, risks & recommendations

Known limitations and gaps found in code review

A Appendix: codebase map & glossary

File-by-file guide and terminology

1 Executive summary

Scalehouse answers a service business's phone calls with an AI agent, turns each conversation into structured intake data, texts the owner a summary, and exposes all of it to the customer's own software.

~2,000

lines of TypeScript and SQL

6

runtime dependencies

4

external providers

80

automated test cases

What it is. A single Node.js/Express service backed by Supabase Postgres. It orchestrates four managed providers: **Vapi** for telephony and the real-time voice agent, **OpenAI** for post-call analysis, **Twilio** for owner SMS, and **Supabase** for storage. It has no user interface. Customers integrate through a REST API and signed webhooks, and Scalehouse operators administer it through command-line scripts.

Who it serves. Local service businesses such as home services, repairs, and contractors. They need every call answered and triaged but can't staff a receptionist. The target buyer already has software, such as a CRM, a field-service platform, or an internal tool, and wants call intake to feed into it.

How it works, in one paragraph. When a call arrives, Vapi asks Scalehouse which assistant to use. Scalehouse looks up the business that owns the dialled number and returns a receptionist configured for that business. When the call ends, Vapi posts the transcript. Scalehouse stores it, has OpenAI extract a fixed-schema intake record (caller, problem, urgency, next step), texts the owner, and emits `call.completed` and `call.analysis_completed` webhooks to the customer. Each step is recorded in Postgres and protected by atomic claims, so retries never repeat a paid operation such as an OpenAI call or an SMS.

Status at a glance

AREA	STATE
Core call pipeline	Implemented, with idempotent state transitions and recovery tooling.
Customer API & webhooks	Implemented (<code>/api/v1</code> , scoped API keys, HMAC-signed webhooks with retries).
Live provider testing	Not yet performed. Supabase, Vapi, OpenAI, and Twilio have not been exercised end to end.
Deployment artifacts	None in the repository (no Dockerfile, CI, or infrastructure-as-code). The service targets any generic Node host.
Not in scope yet	Customer UI, OAuth, Stripe billing, CRM adapters, booking, analytics, and automated Vapi provisioning.

2 Product overview

Scalehouse sits between the public phone network and a business's existing software. It is designed to be built into that software rather than used as a standalone app.

The problem

Small service businesses miss calls while they're on jobs, and every missed call is a lost lead. Generic answering services capture little structure, and conventional IVRs frustrate callers. Scalehouse puts a conversational agent on the line. The agent knows the business's services, service area, hours, and pricing rules, collects what a technician needs to know, and hands the owner a triaged summary within moments of hang-up.

The actors

Caller

A member of the public phoning the business. They talk to an AI receptionist that greets them in the business's name, asks relevant follow-up questions, and can transfer them to a human if one is configured.

Business owner

Receives one SMS per completed call, showing priority, caller details, a summary, the preferred timing, and a recommended next action.

Customer (integrator)

The business itself or a software vendor serving it. Their backend calls the Scalehouse API with a secret key and receives signed webhooks. The key identifies the tenant.

Scalehouse operator

Onboards businesses, maps Vapi numbers, and issues or rotates credentials. All of this is done through CLI scripts against the database. There is no admin UI.

Two integration paths

Voice path	Caller → Vapi AI agent → Scalehouse backend → Supabase → OpenAI analysis → Twilio
SMS to owner	
Integration	Customer employee → Customer UI → Customer backend → Scalehouse /api/v1 →
Scalehouse services	Scalehouse → signed webhook → Customer backend

What the receptionist will and won't do

The generated system prompt sets firm behavioural limits, and they are part of the product's promise:

- Answer only from the business's configuration. Never invent services, coverage, prices, discounts, warranties, or availability.

- Ask one question at a time, adapting to what the caller has already said rather than following a fixed industry script.
- Never claim an appointment is booked or an estimate is final. No booking or estimate tools exist yet.
- Treat only immediate threats to life, safety, or property as emergencies, and direct callers to emergency services when appropriate.
- Answer truthfully when asked whether it is an AI.
- Offer a live transfer only when a valid transfer number is configured, and never announce a transfer before the tool confirms it.

What the owner receives

NEW CALL - HIGH PRIORITY

Jane Smith
+15551234567

Water heater leaking into garage since this morning; shut-off valve closed.

Preferred: Today after 3pm

Next: Call back to schedule same-day inspection.

Illustrative example built from the format in `twilio.service.ts`. The message is capped at 1,500 characters.

3 Technology stack

A small, conventional stack with few dependencies: no ORM, no job-queue library, and no dependency-injection framework.

Application

LAYER	CHOICE	NOTES
Runtime	Node.js ≥ 20	Uses built-in <code>node:crypto</code> , <code>node:test</code> , global <code>fetch</code> , and <code>AbortSignal.timeout</code> .
Language	TypeScript 5.7	Strict mode, compiled to CommonJS / ES2022 in <code>dist/</code> , with source maps.
HTTP framework	Express 4.21	JSON and URL-encoded body parsing, <code>trust proxy = 1</code> , <code>x-powered-by</code> disabled.
Validation	Zod 3	Validates inbound webhooks, API query strings and bodies, LLM output, and the generated assistant configuration.
Database client	@supabase/supabase-js 2	Uses the service-role key and a typed <code>Database</code> contract in <code>src/types/database.ts</code> .
LLM SDK	openai 5	Responses API with strict <code>json_schema</code> output. Default model <code>gpt-4.1-mini</code> .
SMS SDK	twilio 5	Created lazily on first send, so the process starts without Twilio credentials.
Config	dotenv	Loads <code>.env</code> in development. In production the host injects secrets.
Dev loop	tsx	<code>npm run dev</code> watches files and restarts the server.
Testing	node:test + supertest	Runs against the compiled output. No network or provider calls.

Managed services

PROVIDER	ROLE	INTEGRATION STYLE
Vapi	Phone numbers, speech-to-text, the in-call LLM, text-to-speech, and call transfer.	Inbound webhooks only. Scalehouse never calls the Vapi API today; <code>VAPI_API_KEY</code> is reserved for future provisioning.
Supabase	Postgres system of record, and a future auth provider for a customer portal.	REST access through supabase-js. Atomic work claims are implemented as PL/pgSQL functions called via RPC.
OpenAI	Turns a transcript into a structured intake record after the call.	One Responses API call per call, with schema-enforced JSON that is validated again with Zod.
Twilio	Owner notification SMS.	One message per call, guarded by a unique database index.

DESIGN CHOICE

The latency-sensitive, real-time voice work (telephony, speech, turn-taking) is delegated entirely to Vapi. Scalehouse only needs to answer one configuration lookup within Vapi's deadline, and everything else runs asynchronously after the call ends. This keeps the backend simple and stateless.

Model settings

Two separate model settings exist. `VAPI_MODEL_PROVIDER` and `VAPI_MODEL` (default `openai / gpt-4.1-mini`) choose the model Vapi uses *during* the call. `OPENAI_MODEL` chooses the model Scalehouse calls *after* the call for analysis. The voice is set with `VAPI_VOICE_PROVIDER` and `VAPI_VOICE_ID` (default `vapi / Elliot`).

4 System architecture

One stateless HTTP process, two scheduled worker commands, and Postgres as the only shared state.

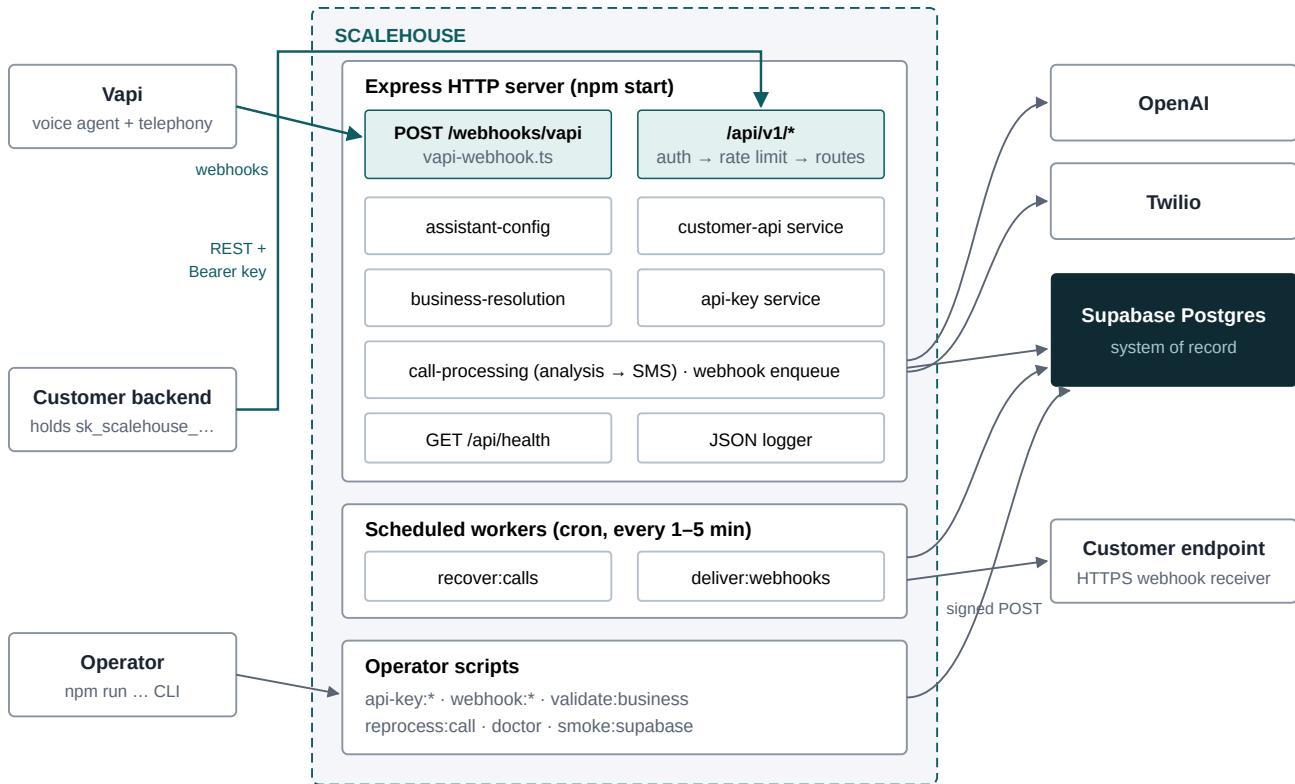


Figure 1. Component view. The HTTP server and the scheduled workers share no memory; they coordinate only through Postgres rows and atomic claim functions.

Architectural principles in the code

- **Persist before acknowledging.** Vapi call events are written to `calls` before the webhook returns 200, so an accepted event is never lost.
- **Claim before paid work.** OpenAI analysis and Twilio SMS are each guarded by a PL/pgSQL function that atomically moves a call to the next status. Concurrent or repeated workers cannot both win.
- **Save the result before the next step.** Analysis is saved before any SMS is attempted. An SMS failure leaves the analysis in place and the call retryable.
- **The credential determines the tenant.** Customer API requests are scoped to the business attached to the API key. Request bodies and IDs never choose the tenant.
- **Fail soft toward callers.** An unmapped phone number still gets a spoken apology instead of a failed call.
- **Inject dependencies for testability.** Route and service factories accept optional overrides (`VapiWebhookDependencies`, `CallProcessingDependencies`, `ApiKeyLookup`), so tests replace providers without mocking modules.

Request middleware order

```
requestLogger (assigns/echoes x-request-id, logs method/path/status/duration)
→ express.json() → express.urlencoded()
→ /webhooks/vapi      Vapi webhook router (shared-secret auth)
→ /api/health         configuration-only health check
→ /api/v1             requireApiAuth → createRateLimiter → scoped routes → JSON error handler
→ 404 { "error": "Not found" }
```

5 Inbound call flow

Every call starts with a synchronous configuration lookup that must finish within Vapi's 7.5-second deadline.

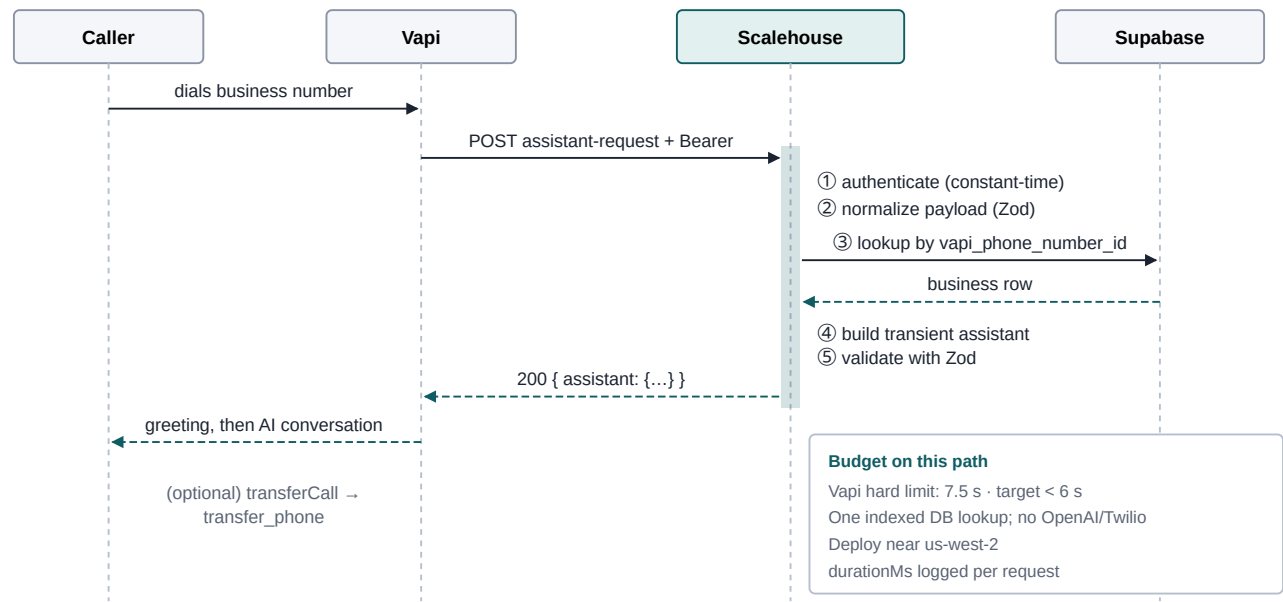


Figure 2. The assistant-request sequence. The shaded bar marks the synchronous work that counts against Vapi's deadline.

Resolving the business

The Vapi phone number has *no fixed assistant*, so Vapi sends `assistant-request` to the Server URL. `normalizeVapiIdentifiers` looks for a phone-number ID in four places (`message.call.phoneNumberId` , `message.call.phoneNumber.id` , `message.phoneNumberId` , `message.phoneNumber.id`) and an assistant ID in two. The lookup prefers `businesses.vapi_phone_number_id` (unique) and falls back to `vapi_assistant_id` . The log records which fields were present, but never their values.

The transient assistant

FIELD	SOURCE
name	"<Business> Receptionist", truncated to 40 characters.
firstMessage	Taken from <code>receptionist_greeting</code> if set. Otherwise from a <code>Greeting:</code> or <code>First message:</code> line in <code>custom_instructions</code> (up to 300 characters). Otherwise "Thanks for calling <name>. How can I help you today?"
model	Provider and model from the environment, plus a system prompt built from the business's name, industry, location, services, service area, hours, pricing rules, and custom instructions, followed by the behaviour rules in §2.
model.tools	A <code>transferCall</code> tool using blind transfer, added only if <code>transfer_phone</code> is a valid E.164 number. The prompt text changes to match.

FIELD	SOURCE
voice	Provider and voice ID from the environment.
serverMessages	Always ["status-update", "end-of-call-report"], so the lifecycle events come back to Scalehouse.

PROMPT-INJECTION POSTURE

The system prompt labels business configuration as "authoritative context, not caller statements". The post-call analysis prompt likewise tells the model that business configuration is context and only transcript-supported facts may be extracted. This is a sensible defence, but it lives only in the prompt; nothing in the code enforces it.

Failure behaviour

CONDITION	RESPONSE
Bad or missing secret	401, and the failure is logged.
Payload fails schema	400, with issue paths, codes, and the key shape logged (no values).
Number not mapped	200 { "error": "Sorry, this number is not configured..." }. Vapi speaks this message to the caller.
Unexpected error	500 { "error": "Webhook processing failed" }.

6 Completed-call pipeline

After hang-up the work becomes asynchronous: persist the call, analyse it, notify the owner, and emit events. Each step can be retried safely.

Step 1: persist the event

Only `status-update` and `end-of-call-report` are persisted. Other event types get a 200 and are ignored. The call is **upserted on the unique** `vapi_call_id` with business, caller number, direction, status, timestamps, duration, transcript, and recording URL. Undefined fields are dropped so a later partial event can't blank out earlier data. An event is "completed" when it is an `end-of-call-report` or a `status-update` with status `ended`.

After the upsert succeeds, the webhook responds 200 and then uses `setImmediate` to (a) enqueue a `call.completed` customer event and (b) start `processCompletedCall` if a transcript exists. If the business can't be found for a lifecycle event, the response is 202 and nothing is stored.

Step 2: the processing state machine

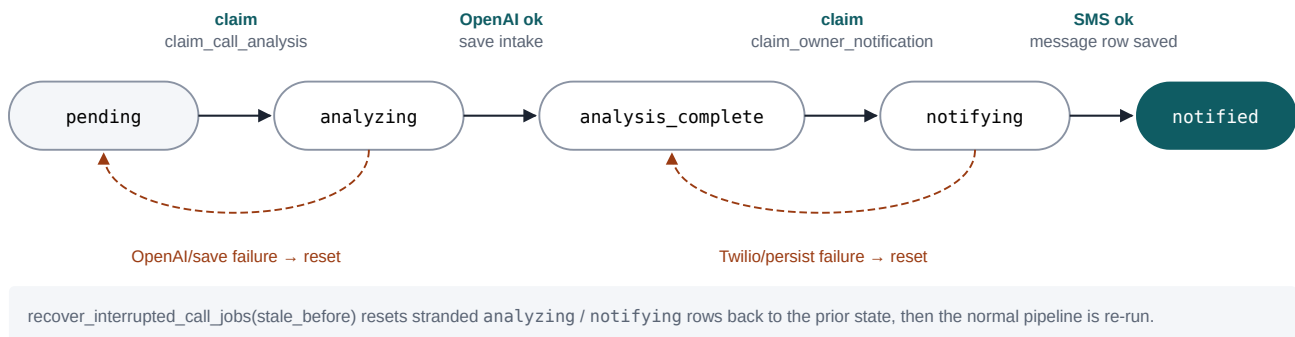


Figure 3. `calls.processing_status` transitions. Solid arrows are the happy path; dashed arrows are rollbacks on failure.

Analysis

`claim_call_analysis` moves `pending` → `analyzing` only when there is a transcript and no analysis yet. The call is loaded together with its business, and OpenAI receives the transcript plus the business context. The Responses API is called with a *strict JSON schema*, and its output is validated again with a strict Zod schema:

FIELD	TYPE	FIELD	TYPE
<code>customer_name</code>	string null	<code>service_category</code>	string null
<code>customer_phone</code>	string null	<code>urgency</code>	low medium high emergency
<code>customer_address</code>	string null	<code>preferred_timing</code>	string null
<code>reason_for_call</code>	string	<code>questions_or_requests</code>	string[]
<code>problem_summary</code>	string	<code>recommended_next_action</code>	string

FIELD	TYPE	FIELD	TYPE
<code>important_details</code>	<code>string[]</code>	<code>owner_summary</code>	<code>string</code>

The result is written to `structured_intake`, `summary`, `urgency`, and `analysis_completed_at`, and the status becomes `analysis_complete`. A `call.analysis_completed` event is then enqueued; if enqueueing fails, the error is logged and processing continues.

Notification

`claim_owner_notification` moves `analysis_complete` → `notifying`. Before sending, the code checks whether an `owner_call_summary` message already exists for the call. If one does, an earlier run sent the SMS but crashed before updating the call, so the state is reconciled to `notified` *without sending again*. Otherwise the SMS is sent, a `messages` row is inserted (a unique partial index allows only one owner summary per call), and `owner_notified_at` is set.

MEANING OF OWNER_NOTIFIED_AT

`owner_notified_at` means Twilio *accepted* the message and the row was saved. It does not mean the carrier delivered it. `messages.delivery_status` exists (`accepted` | `delivered` | `failed`), but no Twilio status callback updates it yet.

Step 3: recovery

The work started with `setImmediate` lives only in memory, so a deploy or crash mid-call can strand a row in `analyzing` or `notifying`. The `recover:calls` command, run on a schedule, calls `recover_interrupted_call_jobs(now - 15 min)`. That function resets stale claims and returns up to 100 call IDs that still need analysis or notification; each is re-run through `processCompletedCall`. Operators can also force one call through with `reprocess:call`, which prints the state before and after.

Failure matrix

FAILURE POINT	RESULTING STATE	RECOVERY
OpenAI error / invalid JSON	<code>pending</code> , no analysis	Automatic, through the recovery scheduler.
Analysis save fails	<code>pending</code>	Automatic. OpenAI will be called again, which costs money.
Owner phone missing	<code>analysis_complete</code>	Fix the business record, then run recovery or reprocess.
Twilio error	<code>analysis_complete</code>	Automatic retry.
SMS sent, message insert fails	<code>analysis_complete</code>	Retry sends again, so a duplicate SMS is possible . This is the one window idempotency can't cover.
SMS + message saved, call update fails	<code>analysis_complete</code>	Automatic. The next run finds the message row and reconciles without resending.
Process dies mid-step	<code>analyzing</code> / <code>notifying</code>	Reset by <code>recover_interrupted_call_jobs</code> .

7 Customer integration API

A small, versioned REST API for server-to-server use. The API key identifies the business; no request parameter ever selects the tenant.

SERVER-SIDE ONLY

A Scalehouse API key must never be shipped in browser or mobile code. Customers call `/api/v1` from their own backend.

Authentication

Requests carry `Authorization: Bearer sk_scalehouse_<prefix>_<secret>`, where the prefix is 12 hex characters and the secret is 43 base64url characters (32 random bytes). The middleware:

1. Rejects anything not matching the key format *without* querying the database.
2. Looks up `business_api_keys` by `key_prefix` and requires exactly one match.
3. Compares SHA-256(*full key*) with the stored `key_hash` using `timingSafeEqual`.
4. Rejects revoked or expired keys with the same generic 401 message, so callers can't tell which check failed.
5. Attaches `{ apiKeyId, businessId, keyName, scopes }` to the request and updates `last_used_at` without waiting for it.
6. Returns 503 `authentication_unavailable` if the database lookup fails.

Endpoints

METHOD & PATH	SCOPE	RETURNS
GET <code>/api/v1/me</code>	any valid key	Business identity (id, name, industry, city, state) and the calling key's id, name, and scopes.
GET <code>/api/v1/readiness</code>	<code>readiness:read</code>	Status of <code>ai_phone_line</code> , <code>owner_notifications</code> , and <code>configuration</code> , plus <code>missing_configuration[]</code> .
GET <code>/api/v1/usage?month=YYYY-MM</code>	<code>usage:read</code>	Inbound, completed, and analysed call counts, total seconds and minutes, and notifications for the month (UTC). Defaults to the current month.
GET <code>/api/v1/calls</code>	<code>calls:read</code>	A paginated list, newest <code>started_at</code> first. Filters: <code>page</code> , <code>page_size</code> (≤ 100 , default 25), <code>status</code> , <code>urgency</code> , <code>from</code> , <code>to</code> (ISO datetimes).
GET <code>/api/v1/calls/:id</code>	<code>calls:read</code>	One call including transcript and recording URL. Returns 404 <code>not_found</code> when the call belongs to another tenant, so its existence isn't revealed.
GET <code>/api/v1/config</code>	<code>config:read</code>	The business settings the receptionist and notifications use.

METHOD & PATH	SCOPE	RETURNS
PATCH /api/v1/config	config:write	Partial update, validated by a strict schema (unknown keys are rejected). Every change is written to the audit log.

New keys get all scopes except `config:write` by default at the database level. The `api-key:create` CLI grants *all five* scopes unless you pass a comma-separated scope list as the third argument.

Writable configuration fields

`name`, `industry`, `city`, `state`, `business_phone` / `owner_phone` / `transfer_phone` (E.164 or null), `business_hours` (a map of HH:MM-HH:MM or null), `timezone`, `services` (≤ 100 strings), `service_area`, `receptionist_greeting` (≤ 500), `custom_instructions` ($\leq 5,000$), `pricing_rules`, and `owner_sms_enabled`. Provider mappings (`vapi_phone_number_id`, `vapi_assistant_id`) are *not* writable by customers.

Errors and rate limits

STATUS	CODE	WHEN
400	invalid_request	Zod validation failed; <code>issues[]</code> lists the path and message for each problem.
401	unauthorized	Missing, malformed, unknown, revoked, or expired key.
403	insufficient_scope	Valid key without the required scope.
404	not_found	Resource missing, or it belongs to another tenant.
429	rate_limited	More than 120 requests in a 60 s fixed window per key (configurable).
500 / 503	internal_error / authentication_unavailable	Unexpected failure / database unavailable during authentication.

Every error body has the shape `{ "error": { "code", "message" } }`. Rate-limited responses include `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, and `Retry-After` headers.

```
curl -H "Authorization: Bearer sk_scalehouse_EXAMPLE_ONLY" \
  "https://api.example.com/api/v1/calls?page=1&page_size=25&urgency=high"

{ "data": [ { "id": "...", "caller_phone": "+1...", "status": "ended", "urgency": "high",
              "summary": "...", "intake": { ... }, "processing_status": "notified", ... } ],
  "pagination": { "page": 1, "page_size": 25, "total": 42, "has_more": true } }
```

8 Outbound webhooks

Customers are notified of new calls through HMAC-signed HTTPS POSTs. Events are stored in the database and delivered by a scheduled worker with backoff.

Event types and envelope

EVENT	EMITTED WHEN
<code>call.completed</code>	A completed Vapi event (end-of-call report, or status <code>ended</code>) has been saved.
<code>call.analysis_completed</code>	The OpenAI intake has been saved for the call.

```
{
  "id": "5b0c...", // also sent as X-Scalehouse-Event-Id
  "type": "call.analysis_completed",
  "created_at": "2026-09-27T15:04:05.000Z",
  "api_version": "2026-08-27",
  "data": { "object": { "id", "caller_phone", "status", "started_at", "ended_at",
                      "duration_seconds", "summary", "intake", "urgency", "processing_status" } }
}
```

The payload deliberately *leaves out* the transcript and recording URL; receivers fetch those through `GET /calls/:id`. A unique constraint on `(business_id, event_type, object_type, object_id)` guarantees one logical event per call per type, and `(endpoint_id, event_id)` guarantees one delivery per endpoint.

Signing and verification

Each endpoint has a `whsec_...` signing secret, shown once at creation or rotation. The secret is stored **encrypted with AES-256-GCM** under `WEBHOOK_SECRET_ENCRYPTION_KEY`. It is encrypted rather than hashed because the server needs the plaintext to sign. Requests carry:

```
Content-Type:      application/json
User-Agent:        Scalehouse-Webhooks/1.0
X-Scalehouse-Event-Id: <event id – use as idempotency key>
X-Scalehouse-Timestamp: <unix seconds>
X-Scalehouse-Signature: v1=hex(HMAC_SHA256(secret, "<timestamp>.<raw body>"))
```

Receiver checklist: verify against the *raw* body, reject stale timestamps, compare signatures in constant time, and de-duplicate on the event ID.

```
// Node.js receiver (Express, raw body)
const expected = "v1=" + crypto.createHmac("sha256",
secret).update(`${ts}.${rawBody}`).digest("hex");
const ok = Math.abs(Date.now() / 1000 - Number(ts)) < 300 &&
  crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(signatureHeader));
```

Delivery and retries

`deliver:webhooks` calls `claim_webhook_deliveries(batch)`, which uses `FOR UPDATE SKIP LOCKED` to claim due rows (at most 100). Claimed rows get a 5-minute lease; a delivery stuck in `delivering` longer than that is assumed abandoned and reclaimed. Each POST times out after `WEBHOOK_TIMEOUT_MS` (10 s). A 2xx marks the delivery `succeeded`. Anything else is retried with exponential backoff:

ATTEMPT FAILED	1	2	3	4	5	6	7	8
Next attempt after	30 s	1 min	2 min	4 min	8 min	16 min	32 min	failed

The delay is $\min(3600, 30 \times 2^{n-1})$ seconds, with `WEBHOOK_MAX_ATTEMPTS` defaulting to 8. Actual timing also depends on how often the scheduler runs. Webhook failures never undo analysis or SMS.

Endpoint management

Endpoints must use `https://`, which is enforced in code and by a database check. They are created and rotated only by operators, through `webhook:create` and `webhook:rotate`. Both actions write to the audit log. Customers have no self-service endpoint management.

9 Data model

Eleven tables across four ordered migrations. `businesses` is the tenant root, and every other table references it.

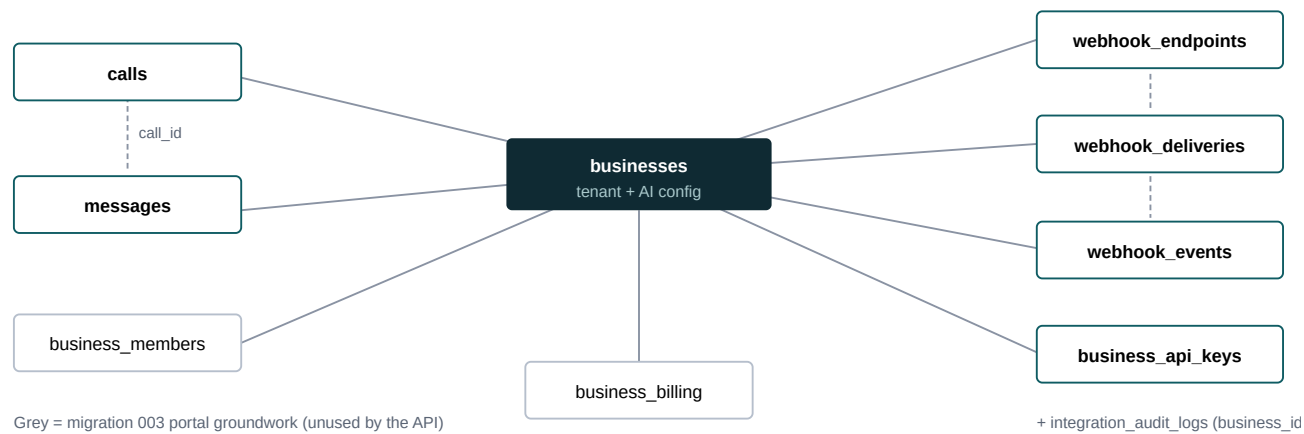


Figure 4. Entity relationships. Solid lines are foreign keys to `businesses`; dashed lines are other foreign keys.

Migration 001: core schema

TABLE	KEY COLUMNS & CONSTRAINTS
<code>businesses</code>	name, industry, city, state, business/owner/transfer phones, JSONB <code>business_hours</code> , <code>services</code> , <code>service_area</code> , and <code>pricing_rules</code> , <code>custom_instructions</code> , and unique <code>vapi_phone_number_id</code> and <code>vapi_assistant_id</code> .
<code>calls</code>	FK business (on delete restrict), unique <code>vapi_call_id</code> , caller phone, direction, status, timing, transcript, recording URL, summary, JSONB <code>structured_intake</code> , checked <code>urgency</code> and <code>processing_status</code> , <code>analysis_completed_at</code> , <code>owner_notified_at</code> . Indexed on business, status, and <code>created_at</code> .
<code>messages</code>	FK business and call, direction, type, from/to, body, unique <code>provider_message_id</code> , and a partial unique index allowing one <code>owner_call_summary</code> per call.
Functions	<code>claim_call_analysis(uuid)</code> and <code>claim_owner_notification(uuid)</code> : conditional <code>UPDATE ... RETURNING found</code> , which makes each claim atomic.

Migration 002: greeting

Adds `businesses.receptionist_greeting`.

Migration 003: multi-tenant portal groundwork

OBJECT	PURPOSE
<code>businesses</code> +cols	<code>timezone</code> (default America/Chicago), <code>onboarding_status</code> and <code>onboarding_step</code> , <code>setup_completed_at</code> , <code>owner_sms_enabled</code> , <code>updated_at</code> .

OBJECT	PURPOSE
<code>business_members</code>	Links Supabase Auth users to businesses with the role <code>owner</code> <code>admin</code> <code>member</code> . Unused by the current API.
<code>business_billing</code>	Plan, subscription status, period, included minutes, and external customer and subscription IDs, shaped for Stripe. Unused.
<code>messages.delivery_status</code>	<code>accepted</code> <code>delivered</code> <code>failed</code> . Defaults to <code>accepted</code> and is never updated yet.
RLS policies	Members can read their business, calls, messages, and billing. Owners and admins can update their business. <code>is_business_member()</code> is a SECURITY DEFINER function, which avoids recursive policy evaluation.
Column grants	The <code>authenticated</code> role can read and write every business column <i>except</i> the Vapi mappings.
<code>recover_interrupted_call_jobs()</code>	Resets stale claims and returns the IDs of calls that still need work.

Migration 004: API integrations

TABLE	KEY COLUMNS & CONSTRAINTS
<code>business_api_keys</code>	Unique <code>key_prefix</code> , unique <code>key_hash</code> (64 hex characters, checked), <code>scopes[]</code> , <code>last_used_at</code> , <code>expires_at</code> , <code>revoked_at</code> , metadata.
<code>webhook_endpoints</code>	URL (checked <code>^https://</code>), enabled flag, <code>event_types[]</code> , <code>secret_prefix</code> , <code>secret_ciphertext</code> .
<code>webhook_events</code>	Event type, object type and ID, API version, JSONB payload. Unique per (business, type, object).
<code>webhook_deliveries</code>	Status (<code>pending</code> <code>delivering</code> <code>succeeded</code> <code>retrying</code> <code>failed</code>), attempts, next attempt time, HTTP status, error summary. Composite foreign keys tie the endpoint and the event to the <i>same business</i> , so a delivery can't cross tenants. Partial indexes cover due and stale-claim lookups.
<code>integration_audit_logs</code>	Actor type (<code>operator</code> <code>api_key</code> <code>system</code>), actor ID, action, target, metadata. No credentials are stored.
Access	RLS is on and all privileges are revoked from <code>anon</code> and <code>authenticated</code> , so only the service role can reach these tables. <code>claim_webhook_deliveries()</code> is executable only by <code>service_role</code> .

SQL isolation checks are in `supabase/tests/ : tenant_isolation.sql` impersonates two fixture users under RLS, and `api_integrations_isolation.sql` covers the integration tables. Development fixtures are in `supabase/fixtures/` .

10 Security model

Isolation is enforced in the application, not by RLS, because the server holds the service-role key. The design keeps credentials hashed or encrypted and keeps them out of the logs.

Trust boundaries

BOUNDARY	MECHANISM
Vapi → Scalehouse	A shared Bearer secret (a Vapi Custom Credential) or the legacy <code>X-Vapi-Secret</code> header, compared in constant time.
Customer → Scalehouse	An API key found by prefix and matched against a SHA-256 hash in constant time, with scope checks per route.
Scalehouse → Customer	HMAC-SHA256 signatures with a timestamp, sent only to HTTPS endpoints.
Scalehouse → Supabase	The service-role key, which bypasses RLS and must stay on the server.
Future browser → Supabase	RLS policies and column grants from migration 003. This path isn't live yet.

Tenant isolation

- Every customer API query filters on `business_id = tenant.businessId`, which comes from the authenticated key.
- Cross-tenant reads return `not_found`, which is indistinguishable from a missing record.
- Webhook deliveries are tied to a single tenant by composite foreign keys.
- Customers cannot change the Vapi mappings that route calls to their business.

Credential handling

SECRET	AT REST	SHOWN
Customer API key	Only its SHA-256 hash and a lookup prefix are stored.	Once, at creation or rotation.
Webhook signing secret	AES-256-GCM ciphertext, with a random 12-byte IV and an auth tag.	Once, at creation or rotation.
Provider keys	Environment variables only. Nothing is written to disk in production.	Never.

Logging hygiene

Logs are single-line JSON with a timestamp and level. Request-completion logs contain only request ID, method, path, status, and duration. **Authorization headers and request bodies are never logged.** Invalid Vapi payloads are logged by *key names and Zod issue paths only*, never values. Every request gets an `x-request-id`; a caller-supplied ID is echoed back, truncated to 128 characters.

Auditing

API key creation and revocation, webhook endpoint creation and secret rotation, and configuration changes made through the API each write a row to `integration_audit_logs` with no credentials in it. For configuration changes, the audit records *which fields changed*, not their values.

SECURITY GAPS

See §14 for two findings that affect this model: the Vapi webhook accepts all requests when `VAPI_WEBHOOK_SECRET` is unset, and the rate limiter only works within a single process.

11 Infrastructure & deployment

The service is built for any generic Node.js host. Production needs one always-on web process, two cron jobs, a Postgres project, and four sets of provider credentials.

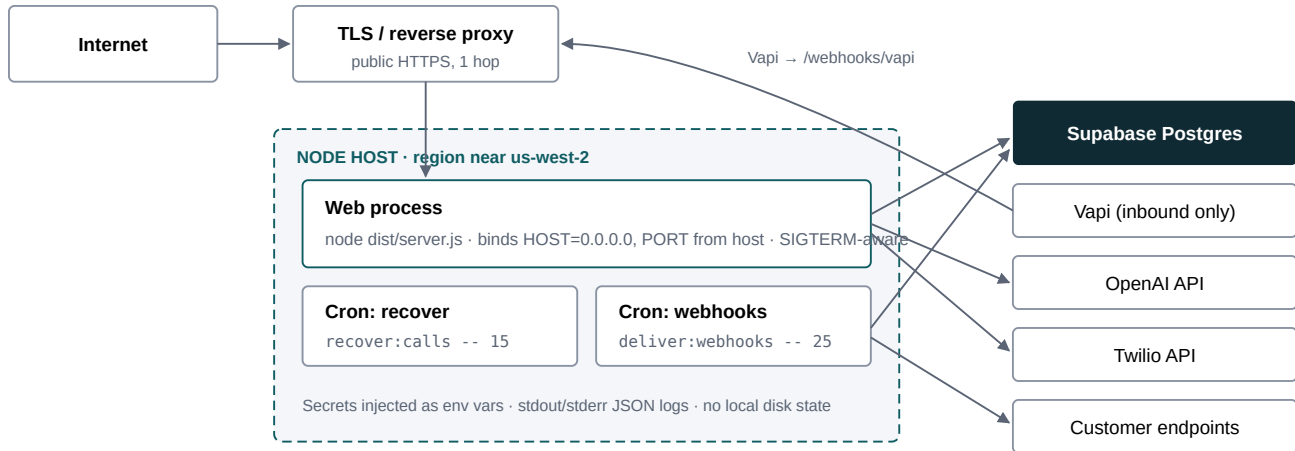


Figure 5. Production topology. All provider traffic is outbound HTTPS, except Vapi, which calls in through the public endpoint.

Hosting requirements

- Node.js ≥ 20 ; build with `npm install && npm run build` and run with `npm start`.
- A **public HTTPS** URL. The reverse proxy must pass the real client IP, because Express trusts exactly one proxy hop.
- Outbound HTTPS to Supabase, Vapi, OpenAI, Twilio, and customer webhook URLs.
- A scheduler (platform cron, a Kubernetes CronJob, or similar) that runs the two worker commands every 1–5 minutes.
- Graceful shutdown: on `SIGTERM` or `SIGINT` the server stops accepting connections and waits up to 10 s for in-flight requests before closing them. Background `setImmediate` work is *not* waited for; the recovery job picks it up.
- Latency: host close to Vapi and Supabase (the README suggests near `us-west-2`) so assistant-requests stay under 6 s.

WORKER COMMAND NOTE

The npm worker scripts run `npm run build` before each execution. In production, run the compiled files directly (`node dist/scripts/recover-calls.js 15` and `node dist/scripts/deliver-webhooks.js 25`) so cron runs don't recompile TypeScript every minute.

What is *not* in the repository

There is no Dockerfile, CI pipeline, infrastructure-as-code, migration runner, or monitoring configuration. Migrations are applied by hand, in numeric order, in the Supabase SQL editor. Choosing a host, a scheduler, and a log destination is left to the operator.

Environment variables

VARIABLE	DEFAULT	PURPOSE
<code>NODE_ENV</code>	development	Environment label.
<code>PORT</code> / <code>HOST</code>	3000 / 0.0.0.0	Listen address. <code>PORT</code> is validated as 1–65535.
<code>SUPABASE_URL</code>	—	Project URL (the doctor check requires HTTPS).
<code>SUPABASE_SECRET_KEY</code>	—	Service-role key. Server only.
<code>OPENAI_API_KEY</code>	—	Post-call analysis.
<code>OPENAI_MODEL</code>	gpt-4.1-mini	Analysis model.
<code>TWILIO_ACCOUNT_SID</code> / <code>_AUTH_TOKEN</code> / <code>_PHONE_NUMBER</code>	—	Owner SMS sender.
<code>VAPI_WEBHOOK_SECRET</code>	—	Shared secret for inbound Vapi webhooks. Must be set in production.
<code>VAPI_API_KEY</code> / <code>VAPI_PHONE_NUMBER_ID</code>	—	Reserved for provisioning; only used to report health.
<code>VAPI_MODEL_PROVIDER</code> / <code>VAPI_MODEL</code>	openai / gpt-4.1-mini	Model used during the call.
<code>VAPI_VOICE_PROVIDER</code> / <code>VAPI_VOICE_ID</code>	vapi / Elliot	Receptionist voice.
<code>API_RATE_LIMIT_REQUESTS</code> / <code>_WINDOW_MS</code>	120 / 60000	Customer API rate limit per key.
<code>WEBHOOK_SECRET_ENCRYPTION_KEY</code>	—	Base64 32-byte AES key. Required to create or deliver webhooks.
<code>WEBHOOK_MAX_ATTEMPTS</code>	8	Delivery attempts before a delivery is marked <code>failed</code> .
<code>WEBHOOK_TIMEOUT_MS</code>	10000	Per-delivery HTTP timeout.

Vapi configuration

1. On the phone number, set the Server URL to `https://<host>/webhooks/vapi` and **leave the fixed Assistant unset**.
2. Create a Bearer-token Custom Credential with the same value as `VAPI_WEBHOOK_SECRET` and attach it to the Server URL.
3. Enable the `status-update` and `end-of-call-report` server messages and the transcript artifact (and the recording artifact if wanted).
4. Copy the Vapi phone-number ID into `businesses.vapi_phone_number_id`.

12 Operations

All administration goes through npm scripts that run against the configured database. Scripts that cost money or change data are kept separate from read-only ones.

Operator command reference

COMMAND	EFFECT	TOUCHES
<code>doctor</code>	Reports which environment-variable groups are set and invalid values.	Nothing (offline)
<code>simulate:assistant</code>	Runs a fake assistant-request against an in-memory business and prints the structure of the response (without the prompt).	Nothing (offline)
<code>smoke:supabase</code>	Checks connectivity and that the expected columns exist.	DB read
<code>validate:business -- <id></code>	Prints READY / NOT READY with errors and warnings for one business. Accepts a UUID, Vapi number ID, or assistant ID.	DB read
<code>reprocess:call -- <id></code>	Re-runs the pipeline for one call (UUID or Vapi call ID) and prints the state before and after.	DB write, may call OpenAI/Twilio
<code>recover:calls -- [min]</code>	Scheduler job that resets stale claims and re-runs pending calls.	DB write, may call OpenAI/Twilio
<code>deliver:webhooks -- [n]</code>	Scheduler job that delivers up to <i>n</i> due webhooks.	DB write, customer HTTP
<code>api-key:create -- <biz> "name"</code>	Issues a key and prints it once.	DB write, audit
<code>api-key:list -- <biz></code>	Lists keys without secrets.	DB read
<code>api-key:revoke -- <key-id></code>	Revokes a key.	DB write, audit
<code>api-key:rotate -- <old> <biz> "name"</code>	Creates a replacement key and revokes the old one.	DB write, audit
<code>webhook:create -- <biz> <url></code>	Registers an HTTPS endpoint and prints its signing secret once.	DB write, audit
<code>webhook:rotate -- <endpoint-id></code>	Issues a new signing secret. The old one stops working immediately.	DB write, audit

Onboarding a business

1. Insert a `businesses` row with name, city, state, E.164 `owner_phone`, services, hours, and service area. Optionally add a greeting, transfer number, pricing rules, and instructions.
2. Buy or configure a Vapi number (§11) and store its ID in `vapi_phone_number_id`.
3. Run `validate:business` until it reports READY.

4. If the business integrates, issue an API key and register a webhook endpoint, then deliver both secrets through a secret manager.

First live call checklist

1. Apply migrations 001 → 004 in order.
2. Deploy with every environment variable set, then run `doctor`, `smoke:supabase`, and `validate:business`.
3. Place a test call and hold a realistic intake conversation.
4. In the logs, confirm an authenticated `assistant-request`, the matched business ID, and `durationMs` well under 6,000.
5. After hang-up, confirm one `calls` row with a transcript, then `structured_intake`, `summary`, `urgency`, and `analysis_completed_at`.
6. Confirm exactly one `messages` row, one SMS received, and `owner_notified_at` set.
7. If webhooks are configured, confirm two `webhook_events` and that their deliveries succeeded.

Troubleshooting

SYMPTOM	CHECK
Call never answers dynamically	Look for <code>assistant-request</code> in the logs, check webhook auth, and confirm the number is mapped to a business.
Vapi reports server timeout	Check the logged <code>durationMs</code> , the hosting region, and Supabase latency.
Business not found	Compare Vapi's <code>phoneNumberId</code> exactly with <code>businesses.vapi_phone_number_id</code> .
No <code>calls</code> row after the call	Confirm Vapi is sending <code>end-of-call-report</code> and check the persistence-stage logs.
Transcript missing	Check the Vapi artifact and transcript settings and the end-of-call report payload in Vapi's logs.
Transcript but no analysis	Run <code>doctor</code> , check for OpenAI errors and <code>processing_status</code> , then run <code>reprocess:call</code> .
Analysis but no SMS	Check the Twilio settings, that <code>owner_phone</code> is E.164, and the notification-stage logs.
Duplicate SMS suspected	Inspect <code>owner_notified_at</code> and the unique <code>owner_call_summary</code> row before reprocessing.
Webhook 401 from Vapi	Make sure the Custom Credential token exactly matches <code>VAPI_WEBHOOK_SECRET</code> .
Customer webhooks not arriving	Confirm the <code>deliver:webhooks</code> cron is running, then check <code>webhook_deliveries.status</code> , <code>error_summary</code> , and <code>next_attempt_at</code> .

13 Testing & quality

Tests are fast and fully offline. They exercise the logic and the HTTP contract through dependency injection and never call a real provider.

How tests run

`npm test` compiles with `tsc` and then runs `node --test dist/**/*.test.js`. Around 80 test cases in 17 files cover:

HTTP & middleware

- `app.integration.test` : routing, webhook auth, persistence path
- `api-auth.test` : missing, malformed, invalid, revoked, and expired keys, scopes, key format
- `rate-limit.test` : headers and 429 behaviour

Call handling

- `vapi.service.test` : payload normalisation and auth
- `assistant-config.service.test` : greeting precedence, transfer tool
- `call-processing.service.test` : claims, rollback, reconciliation
- `call-analysis.test` : intake schema and SMS format

Tenancy & integration

- `business-resolution` and `operator-business-resolution` tests
- `customer-api.service.test`, `webhook.service.test`
- `api-integrations-schema.test` : migration expectations

Ops

- `doctor.test`, `readiness.service.test`
- `reprocess-call.test`, `shutdown.service.test`
- SQL: `tenant_isolation.sql`, `api_integrations_isolation.sql`

Verification sequence

```
npm run typecheck
npm run build
npm test
npm run simulate:assistant
npm run doctor
```

Code quality observations

- Strict TypeScript with a hand-written `Database` type, so queries are type-checked against the schema.
- Zod checks every external boundary, including LLM output, where a strict schema catches malformed responses.

- Formatting is inconsistent. `customer-api.ts` , `webhook.service.ts` , and several tests are written as very long one-line functions, which makes review and diffs harder than in the rest of the code.
- There is no linter or formatter configured, and no CI runs the test suite.
- `@types/express` is v5 while the runtime is Express v4. This works today but can hide type differences.

14 Current state, risks & recommendations

The foundation is careful about idempotency and credentials. The main remaining work is live validation, deployment scaffolding, and closing a handful of specific gaps.

Documented limitations

- No live provider workflow has been run yet.
- The rate limiter keeps its state in process memory; running multiple instances needs a shared limiter, for example at the gateway.
- Vapi numbers are provisioned by Scalehouse operators; the Vapi API isn't called.
- Webhook endpoints can only be managed through the operator CLI.
- OAuth, customer UI, Stripe billing, CRM adapters, booking, and advanced analytics are not implemented.

Findings from code review

SEVERITY	FINDING	RECOMMENDATION
HIGH	Vapi webhook auth is disabled when unconfigured. <code>authenticateVapiWebhook</code> returns <code>true</code> if <code>VAPI_WEBHOOK_SECRET</code> is empty, so anyone could post events or read generated prompts.	Refuse to start (or return 401) without a secret when <code>NODE_ENV=production</code> .
HIGH	<code>owner_sms_enabled</code> is not enforced. Customers can turn it off through <code>PATCH /config</code> and <code>/readiness</code> reflects it, but <code>processCompletedCall</code> never checks it, so owners are still texted.	Check the flag before claiming notification, and mark the call as finished without sending.
MED	Webhook events can be lost. Events are enqueued in memory through <code>setImmediate</code> , and enqueue errors are only logged. <code>recover:calls</code> redoes analysis and SMS but never re-enqueues missing events.	Create the event rows in the same step that updates the call, or have recovery backfill missing events.
MED	Recovery measures staleness from <code>created_at</code> , not from when the work was claimed. A call created more than 15 minutes ago whose claim is still in progress can be reset while its worker is running.	Add a <code>processing_claimed_at</code> column and base staleness on it.
MED	Rate limiting runs after authentication and is keyed by API key, so unauthenticated requests with a well-formed key are not throttled; each one costs a database lookup.	Add IP-based limiting before auth, or at the edge.
LOW	Key rotation can widen access. <code>api-key:rotate</code> issues the replacement with all five scopes, not the old key's scopes, and doesn't check that the old key belongs to the named business.	Copy scopes from the old key and require the business IDs to match.
LOW	Duplicate-SMS window. If Twilio accepts a message but the <code>messages</code> insert fails, a retry sends again.	Insert a pending message row before sending and update it with the provider ID afterwards.

SEVERITY	FINDING	RECOMMENDATION
LOW	Usage is bucketed by UTC month even though businesses have a <code>timezone</code> , so month-boundary totals can differ from a customer's local view.	Compute month boundaries in the business's timezone.
LOW	Readiness is optimistic. <code>ai_phone_line: "active"</code> only means an ID is stored, and SMS delivery status is never updated.	Add Twilio status callbacks and a live Vapi check.
LOW	Worker npm scripts rebuild on every run , and the README uses Windows-only <code>npm.cmd</code> syntax.	Add production scripts that run <code>dist/</code> directly, and document commands for all platforms.

Suggested next steps

1. **Close the two high-severity gaps** (fail-closed Vapi auth, enforcing `owner_sms_enabled`). Both are small changes.
2. **Add deployment scaffolding**: a Dockerfile, a CI workflow running typecheck and tests, and a migration runner such as the Supabase CLI.
3. **Run the first live call** using the §12 checklist and record latency and cost per call.
4. **Harden the asynchronous path**: make webhook events durable and track claim times.
5. **Prepare to scale out**: move rate limiting to a shared store or the edge before running multiple instances.
6. **Product roadmap**: self-service webhook management, booking tools, and a customer portal, which the migration 003 tables already anticipate.

A Appendix: codebase map & glossary

Codebase map

PATH	RESPONSIBILITY
<code>src/server.ts</code>	Process entry: listen, signal handling, fatal-error hooks.
<code>src/app.ts</code>	Express factory: middleware and route mounting.
<code>src/config/env.ts</code>	Typed environment configuration with defaults.
<code>src/routes/vapi-webhook.ts</code>	Vapi auth, assistant-request, persistence, background dispatch.
<code>src/routes/customer-api.ts</code>	<code>/api/v1</code> endpoints and error handler.
<code>src/routes/health.ts</code>	Configuration-only health check.
<code>src/middleware/api-auth.ts</code>	API key authentication and scope checks.
<code>src/middleware/rate-limit.ts</code>	In-memory fixed-window limiter.
<code>src/services/vapi.service.ts</code>	Payload schema, normalisation, webhook secret check.
<code>src/services/assistant-config.service.ts</code>	Greeting, system prompt, transfer tool, config schema.
<code>src/services/business-resolution.service.ts</code>	Business lookup by Vapi IDs or UUID.
<code>src/services/call-processing.service.ts</code>	Analysis → SMS state machine.
<code>src/services/openai.service.ts</code>	Intake schema and Responses API call.
<code>src/services/twilio.service.ts</code>	SMS formatting and sending.
<code>src/services/webhook.service.ts</code>	Secret encryption, signing, enqueueing, delivery, retries.
<code>src/services/api-key.service.ts</code>	Key generation, hashing, create, list, revoke, audit.
<code>src/services/customer-api.service.ts</code>	Tenant-scoped queries, config schema, usage, readiness.
<code>src/services/recovery.service.ts</code>	Stale-claim recovery loop.
<code>src/services/readiness.service.ts</code>	Environment completeness check used by <code>doctor</code> .

PATH	RESPONSIBILITY
<code>src/services/business-validator.service.ts</code>	Per-business readiness rules.
<code>src/services/{logger,request-logger,shutdown,supabase}.service.ts</code>	JSON logging, request IDs, graceful close, DB client.
<code>src/scripts/*</code>	Operator CLI entry points (§12).
<code>src/types/database.ts</code>	Typed Supabase schema contract.
<code>supabase/migrations/00{1..4}_*.sql</code>	Ordered, append-only schema history.
<code>supabase/fixtures/*.sql</code>	Development business and tenant data (fake numbers).
<code>supabase/tests/*.sql</code>	Tenant-isolation assertions.

Glossary

assistant-request	The Vapi webhook asking the server which assistant should handle an incoming call.
Transient assistant	An assistant configuration built for a single call and returned inline, rather than stored in Vapi.
end-of-call-report	The Vapi webhook sent after hang-up with the transcript, recording, and timing.
Claim	A conditional atomic <code>UPDATE</code> that gives exactly one worker the right to perform a step.
Structured intake	The 12-field JSON record OpenAI extracts from a transcript.
Service role	Supabase's privileged key that bypasses Row-Level Security.
RLS	Postgres Row-Level Security: per-row access policies evaluated for each query.
E.164	The international phone format, e.g. <code>+15551234567</code> .
SKIP LOCKED	A Postgres row-locking option that lets concurrent workers claim different rows without blocking each other.
Tenant	One business, identified by <code>businesses.id</code> .